

IMC Prosperity Challenge

Manual Trading: Sealed-Bid Auction Strategy

Ember Mushroom & Dryland Flax

April 18, 2026

Contents

1	Problem Context	2
2	Mechanics: Price-Time Priority Auction	2
2.1	Matching Algorithm	2
2.2	Profit Formula	2
2.3	Cumulative Supply and Demand Curves	3
3	Ember Mushroom Analysis	3
3.1	Stale Order Book	3
3.2	Finding the Stale-Book Clearing Price	3
3.3	Stale-Book Match Trace	4
3.4	Profit Calculation per Ask Level	4
3.5	Competition-Adjusted Strategy	4
4	Dryland Flax Analysis	5
4.1	Stale Order Book	5
4.2	Cumulative Demand and Supply	5
4.3	Stale-Book Match Trace and Residual	5
4.4	The Price-Priority Trick	6
4.5	Profit Calculation	6
5	Validation Code	7
6	Supply & Demand Curves (Visualisation)	8
7	Final Orders Summary	9

1 Problem Context

As a special welcoming event, the Intarians hold a one-time exchange auction. Two tradable goods are available:

- **Ember Mushroom** – fee 0.05 XIRECs per unit on buy *and* on sell; guaranteed exit price **20 XIRECs/unit**.
- **Dryland Flax** – no explicit fee stated; guaranteed exit price **30 XIRECs/unit**.

Each participant may submit **one order per good** (buy or sell, at a chosen price and volume). The exchange clears using **price-time priority**: the highest-priced bids match against the lowest-priced asks first; ties at the same price are resolved by submission time.

The order books are *stale* – they represent orders placed by other (NPC) participants before the auction round. Our order is added on top and the full book then clears simultaneously.

Objective

Identify bid price and volume for each good that maximises expected profit, accounting for:

1. residual supply after stale-book matching,
2. guaranteed exit (sell) prices,
3. trading fees,
4. competition risk (other teams targeting the same price levels).

2 Mechanics: Price-Time Priority Auction

2.1 Matching Algorithm

Let $\mathcal{B} = \{(p_i^B, v_i^B)\}$ be the sorted bid side (descending by price) and $\mathcal{A} = \{(p_j^A, v_j^A)\}$ the sorted ask side (ascending by price). The auction iterates:

$$\text{Fill}_k = \min\left(v_{\text{bid}}^{\text{remaining}}, v_{\text{ask}}^{\text{remaining}}\right) \quad \text{while } p_{\text{bid}} \geq p_{\text{ask}}$$

Each matched unit trades at the **ask price** (the sell side sets the price in standard double-auction models). When the highest remaining bid price falls below the lowest remaining ask price, matching stops and residual orders remain unfilled.

2.2 Profit Formula

For a *buy* order that fills at execution price p^* and exits at guaranteed price P_{exit} :

$$\Pi = (P_{\text{exit}} - p^* - F_{\text{buy}} - F_{\text{sell}}) \times V_{\text{filled}} \quad (1)$$

where $F_{\text{buy}}, F_{\text{sell}}$ are per-unit fees and V_{filled} is the actual filled volume.

2.3 Cumulative Supply and Demand Curves

Define:

$$D(p) = \sum_{v_i^B \geq p} v_i^B \quad (\text{cumulative demand at price } p) \quad (2)$$

$$S(p) = \sum_{v_j^A \leq p} v_j^A \quad (\text{cumulative supply at price } p) \quad (3)$$

The **clearing price** p^* satisfies $D(p^*) = S(p^*)$, or equivalently is the price at which the excess demand $D(p) - S(p)$ crosses zero from above.

3 Ember Mushroom Analysis

3.1 Stale Order Book

Table 1: Ember Mushroom stale order book

BID (buy orders)		ASK (sell orders)	
Volume	Price	Price	Volume
43,000	20	12	20,000
17,000	19	13	25,000
6,000	18	14	35,000
5,000	17	15	6,000
10,000	16	16	5,000
5,000	15	17	0
10,000	14	18	10,000
7,000	13	19	12,000

3.2 Finding the Stale-Book Clearing Price

Table 2: Cumulative demand $D(p)$ and supply $S(p)$ for Ember Mushroom

Price p	$D(p)$	$S(p)$	Excess Demand $D - S$
12	103,000	20,000	+83,000
13	103,000	45,000	+58,000
14	96,000	80,000	+16,000
15	86,000	86,000	0
16	81,000	91,000	-10,000
17	71,000	91,000	-20,000
18	66,000	101,000	-35,000
19	60,000	113,000	-53,000
20	43,000	113,000	-70,000

$$p_{\text{EM, stale}}^* = \boxed{15} \quad D(15) = S(15) = 86,000$$

3.3 Stale-Book Match Trace

The matching engine processes in price priority order:

Bid	Ask	Bid vol	Ask vol	Fill
43k @ 20	20k @ 12	43,000	20,000	20,000
23k @ 20	25k @ 13	23,000	25,000	23,000
17k @ 19	2k @ 13	17,000	2,000	2,000
15k @ 19	35k @ 14	15,000	35,000	15,000
6k @ 18	20k @ 14	6,000	20,000	6,000
5k @ 17	14k @ 14	5,000	14,000	5,000
10k @ 16	9k @ 14	10,000	9,000	9,000
1k @ 16	6k @ 15	1,000	6,000	1,000
5k @ 15	5k @ 15	5,000	5,000	5,000
Total filled: 86,000				

Residual after stale match:

- **Asks remaining** (our opportunity): 5,000 @ 16 — 10,000 @ 18 — 12,000 @ 19
- **Bids remaining:** 10,000 @ 14 — 7,000 @ 13 (cannot cross remaining asks)

Total residual supply available = 5,000 + 10,000 + 12,000 = **27,000** units.

3.4 Profit Calculation per Ask Level

Using Equation (1) with $P_{\text{exit}} = 20$, $F_{\text{buy}} + F_{\text{sell}} = 0.10$:

Table 3: Ember Mushroom: profit by execution price

Ask price p^*	Volume	$P_{\text{exit}} - p^*$	– fees	Profit/unit
16	5,000	$20 - 16 = 4.00$	–0.10	3.90
18	10,000	$20 - 18 = 2.00$	–0.10	1.90
19	12,000	$20 - 19 = 1.00$	–0.10	0.90

Maximum theoretical profit (capture all residual supply):

$$\Pi_{\text{max}} = (3.90 \times 5,000) + (1.90 \times 10,000) + (0.90 \times 12,000) = 49,300 \text{ XIRECs}$$

3.5 Competition-Adjusted Strategy

Multiple teams will identify the same residual supply and bid into the 15–19 range. This pushes the effective clearing price *upward*, reducing per-unit profit but increasing fill certainty.

Scenario analysis for a 10,000-unit order at bid price 19:

Table 4: Ember Mushroom: profit scenarios vs. effective clearing price

Effective clearing p^*	Profit/unit	Volume filled	Total profit
16	3.90	10,000	39,000
17	2.90	10,000	29,000
18	1.90	10,000	19,000
19	0.90	10,000	9,000

All scenarios are profitable. Bidding at 19 guarantees the highest fill priority among our orders while keeping $\Pi > 0$ even in the worst case.

Action: **BUY** $p = 19$ $V = 10,000$ $\Rightarrow$ $\Pi \in [9,000, 39,000]$ XIRECs

4 Dryland Flax Analysis

4.1 Stale Order Book

Table 5: Dryland Flax stale order book

BID		ASK	
Volume	Price	Price	Volume
30,000	30	28	40,000
5,000	29	31	20,000
12,000	28	32	20,000
28,000	27	33	30,000

4.2 Cumulative Demand and Supply

Table 6: Dryland Flax: cumulative supply and demand

p	$D(p)$	$S(p)$	Excess Demand
27	75,000	0	+75,000
successbg 28	47,000	40,000	+7,000
29	35,000	40,000	−5,000
30	30,000	40,000	−10,000

The stale-book clearing price is **28**, with excess demand of 7,000 units (demand exceeds supply).

4.3 Stale-Book Match Trace and Residual

Matching proceeds highest-bid-first:

Bid	Ask	Bid rem.	Ask rem.	Fill
30k @ 30	40k @ 28	30,000	40,000	30,000
5k @ 29	10k @ 28	5,000	10,000	5,000
12k @ 28	5k @ 28	12,000	5,000	5,000
7k @ 28	<i>no ask</i>	7,000	—	0
Total filled:				40,000

Residual:

- Asks: 20k @ 31, 20k @ 32, 30k @ 33 (all unprofitable: $31 > 30 = P_{\text{exit}}$)
- Bids: 7k @ 28, 28k @ 27 (unfilled, could not cross remaining asks)
- **No profitable residual ask supply exists at or below 29.**

4.4 The Price-Priority Trick

At first glance, Dryland Flax looks unattractive. But the key insight is in the *ordering* of the 12,000 stale bids sitting at price 28:

Insight: A new bid at price **29** has higher price priority than existing bids at **28**. In the auction sequence, it fills *before* the 12k stale bids at 28.

Inserting our bid of 5,000 units at price 29, the match sequence becomes:

Bid	Ask	Fill	Cumul. filled
30k @ 30 (stale)	40k @ 28	30,000	30,000
5k @ 29 (stale)	10k @ 28	5,000	35,000
infoblu!12 5k @ 29 (MY ORDER)	5k @ 28	5,000	40,000
12k @ 28 (stale)	<i>no asks left</i>	0	40,000

Our 5,000 units fill at $p^* = 28$. The stale 12k bid at 28 gets zero fill.

4.5 Profit Calculation

$$\Pi_{\text{DF}} = (P_{\text{exit}} - p^* - F) \times V = (30 - 28 - 0) \times 5,000 = \mathbf{10,000 \text{ XIRECs}}$$

Action: **BUY** $p = 29$ $V = 5,000$ (executes at 28) $\Pi = 10,000 \text{ XIRECs}$

Only 5,000 units are available at this slot. If another team submits the same order at price 29 *before* us, price-time priority means they fill first and we receive nothing. This is a high-alpha, non-zero-risk play. Volume should be capped at 5,000.

5 Validation Code

The following Python script reproduces all computations above.

Listing 1: Order book analysis and match simulation

```

1  # Order book data
2  em_asks = [(12,20000),(13,25000),(14,35000),(15,6000),
3            (16,5000),(17,0),(18,10000),(19,12000)]
4  em_bids = [(20,43000),(19,17000),(18,6000),(17,5000),
5            (16,10000),(15,5000),(14,10000),(13,7000)]
6
7  df_asks = [(28,40000),(31,20000),(32,20000),(33,30000)]
8  df_bids = [(30,30000),(29,5000),(28,12000),(27,28000)]
9
10 def clearing_analysis(bids, asks):
11     """Compute cumulative D(p) and S(p) for each price level."""
12     all_prices = sorted(set(p for p, _ in bids+asks))
13     for p in all_prices:
14         d = sum(v for price, v in bids if price >= p)
15         s = sum(v for price, v in asks if price <= p)
16         print(f"p={p}: D={d:}, S={s:}, excess={d-s:+},")
17
18 def simulate_match(bids_raw, asks_raw):
19     """Price-time priority match; returns (rem_asks, rem_bids)."""
20     bids = [[p, v] for p, v in sorted(bids_raw, key=lambda x: -x
21                                     [0])]
22     asks = [[p, v] for p, v in sorted(asks_raw, key=lambda x: x
23                                     [0])]
24     bi, ai = 0, 0
25     while bi < len(bids) and ai < len(asks):
26         if bids[bi][0] < asks[ai][0]:
27             break
28         fill = min(bids[bi][1], asks[ai][1])
29         bids[bi][1] -= fill
30         asks[ai][1] -= fill
31         if bids[bi][1] == 0: bi += 1
32         if asks[ai][1] == 0: ai += 1
33     return [(p,v) for p,v in asks if v>0], [(p,v) for p,v in bids
34         if v>0]
35
36 # Ember Mushroom
37 print("=== EMBER MUSHROOM ===")
38 clearing_analysis(em_bids, em_asks)
39 em_rem_asks, em_rem_bids = simulate_match(em_bids, em_asks)
40 print("Residual asks:", em_rem_asks)
41
42 exit_em, fee_em = 20, 0.10
43 for p, v in em_rem_asks:
44     profit = (exit_em - p - fee_em) * v
45     print(f" {v:}, @ {p} -> profit/unit={exit_em-p-fee_em:.2f},

```

```

43         total={profit:,.0f}")
44 # My order: BUY @ 19, 10000
45 my_p, my_v = 19, 10_000
46 print(f"\nMy_EM_order: BUY_{my_v:}, @_{my_p}")
47 for scenario_clear in [16, 17, 18, 19]:
48     pu = exit_em - scenario_clear - fee_em
49     print(f"    If_clearing={scenario_clear}: profit={pu:.2f} x_{my_v:}, =_{pu*my_v:,.0f}")
50
51 # Dryland Flax
52 print("\n=== DRYLAND_FLAX ===")
53 clearing_analysis(df_bids, df_asks)
54 df_rem_asks, df_rem_bids = simulate_match(df_bids, df_asks)
55 print("Residual_asks:", df_rem_asks)
56
57 exit_df = 30
58 # Insert my bid at 29 before the 12k stale bid at 28
59 bids_with_mine = df_bids + [(29, 5000)]
60 df2_rem_asks, df2_rem_bids = simulate_match(bids_with_mine, df_asks)
61 print("Residual_asks_(with_my_29-bid):", df2_rem_asks)
62 print(f"My_DF_order: BUY_5000 @ 29 -> fills at 28 -> profit={ (exit_df-28)*5000:,.0f}")

```

6 Supply & Demand Curves (Visualisation)

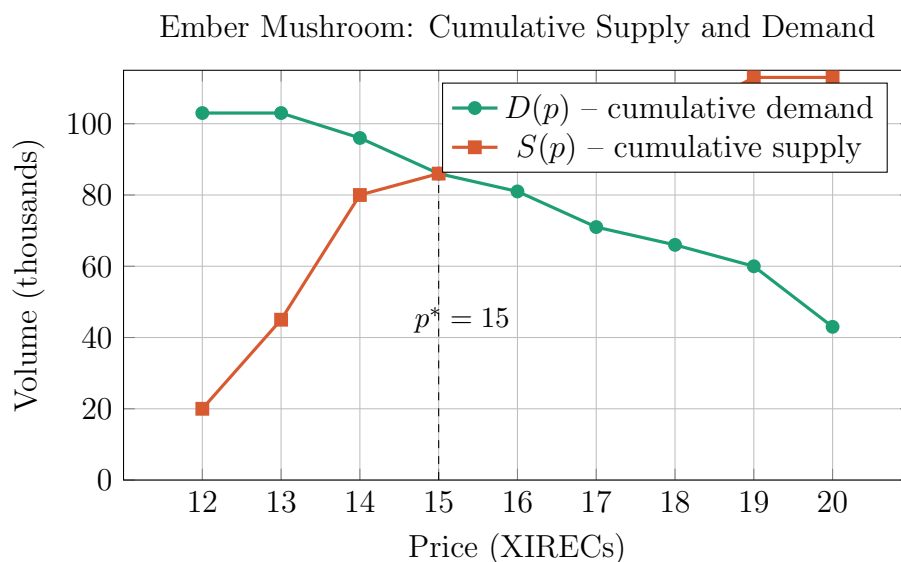


Figure 1: Clearing price for Ember Mushroom stale book: $p^* = 15$, matched volume = 86,000

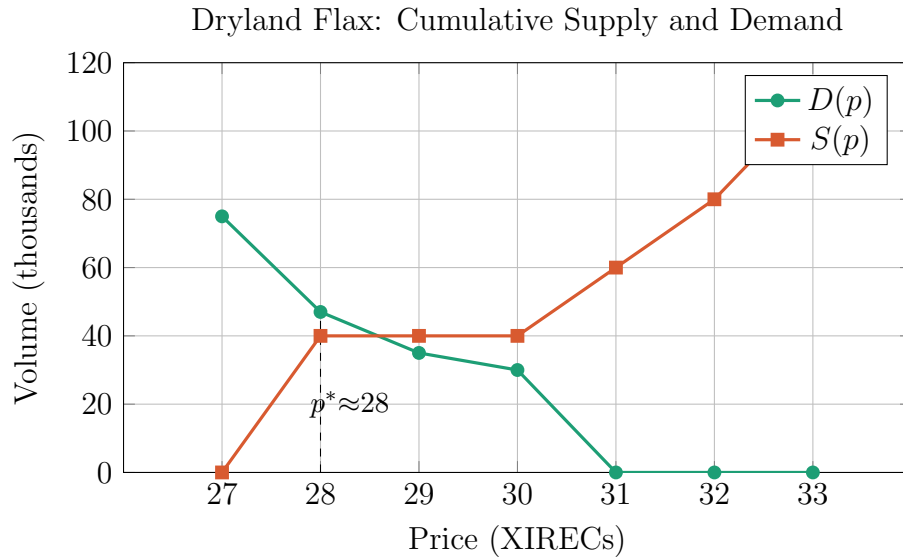


Figure 2: Dryland Flax: clearing occurs at $p^* = 28$, with excess demand of 7,000 units

7 Final Orders Summary

Table 7: Submitted orders

Good	Action	Bid Price	Volume	Expected Profit
Ember Mushroom	BUY	19 XIRECs	10,000	9,000 – 39,000 XIRECs
Dryland Flax	BUY	29 XIRECs	5,000	10,000 XIRECs
Combined expected			19,000	19,000 – 49,000 XIRECs

Reasoning Summary

1. **Ember Mushroom:** The stale book matches internally at 15, leaving 27k units of profitable residual supply (prices 16–19). Bidding at 19 maximises fill priority. With guaranteed exit at 20 and fees of 0.10/unit, every scenario where we fill is profitable.
2. **Dryland Flax:** No residual supply exists below the 30 XIREC exit price after the stale book matches. The opportunity is to exploit price-time priority: bidding at 29 (not 28) places our order ahead of 12,000 stale bids at 28, capturing the final 5,000 units of the 40k @ 28 ask level at execution price 28, for a clean $30 - 28 = 2$ XIREC/unit margin.
3. **Competition:** For EM, 27k residual supply provides ample room for multiple participants; the risk is modest. For DF, only 5k units are available at this price point, making submission timing critical.

Appendix: Key Formulae Reference

$$\text{Profit per unit} = P_{\text{exit}} - p^* - F_{\text{total}} \quad (4)$$

$$\text{Total profit} = (P_{\text{exit}} - p^* - F_{\text{total}}) \times V_{\text{filled}} \quad (5)$$

$$D(p) = \sum_{i: p_i^B \geq p} v_i^B \quad (6)$$

$$S(p) = \sum_{j: p_j^A \leq p} v_j^A \quad (7)$$

$$p^* = \arg \min_p |D(p) - S(p)| \quad (8)$$

where:

- p^* = execution (clearing) price
- P_{exit} = guaranteed sell price post-auction
- $F_{\text{total}} = F_{\text{buy}} + F_{\text{sell}}$ = total round-trip fee per unit
- V_{filled} = actual filled volume